

API Connector

for Odoo 18

Outbound REST, JSON-RPC, GraphQL and SOAP calls from Odoo

Version 18.0.1.0.0

IROKOO

sbt@irokoo.fr | https://www.irokoo.fr

Version History

Version	Date	Changes
18.0.1.0.0	2026-10	Documentation in English and French, screenshots of version 18

Table of Contents

1 Overview 3
1.1 What the module does 3
1.2 Features 3
1.3 Module sheet 3
1.4 Menus 3
2 Installation 4
2.1 Prerequisites 4
2.2 Install the module 4
2.3 What the installation adds 4
3 Getting Started: a First Call 4
4 Collections 5
4.1 Collection Settings 5
4.2 Headers and Variables 6
5 Endpoints 7
5.1 Basic Configuration 7
5.2 Headers 8
5.3 Request Body 8
5.4 Response and Statistics 9
6 Authentication 9
6.1 Inherit or define 9
6.2 Methods 9
6.3 OAuth 2.0 10
7 Variables 10
8 Automatic Execution 10
8.1 From a scheduled action 10
8.2 Reading the result 11
9 Monitoring 12
9.1 Call log 12
9.2 Retention 13
10 Export and Import 13
10.1 Export 13
10.2 Import 13
10.3 Postman collections 13
11 Access Rights 14
12 Technical Reference 14
12.1 Models 14
12.2 Useful methods 15
12.3 Deleting a critical endpoint 15

1 Overview

1.1 What the module does

API Connector lets Odoo call external APIs. Each call is described in a form, the way Postman does it: address, HTTP method, authentication, headers, request body. Run it with one click to test it, then hand it to a scheduled action or a server action so that it runs on its own. Every call leaves a trace in a log.

The module is the outbound counterpart of Advanced JSON-RPC API, which exposes Odoo data to inbound calls.

1.2 Features

Feature	What it does
Collections	Group the endpoints of one API and share the base URL, authentication, headers and variables.
Endpoints	One configured call: API type, method, path, headers, body, timeout, SSL verification.
API types	REST, JSON-RPC, GraphQL, SOAP/XML.
Authentication	None, Basic Auth, Bearer Token, API Key in a header, OAuth 2.0 (client credentials).
Variables	`{name}` placeholders replaced in the URL path and in the request body.
Execution	Button in the form or in the list, and call from code in a scheduled action or a server action.
Monitoring	Log of every call (status, duration, data sent and received), success rate per endpoint and per collection.
Export and import	Configuration as JSON, and import of Postman collections v2.0 to v2.2.

1.3 Module sheet

Property	Value
Technical name	`iro_api_connector`
Version	18.0.1.0.0
Odoo version	18
License	OPL-1
Author	IROKOO
Odoo dependencies	`base`, `web`
Python libraries	`requests`, `jsonschema`

1.4 Menus

The module adds the **API Connector** application:

Menu	Entry	Content
Configuration	Collections	Collections and their shared settings
	Endpoints	All configured calls

	Import Configuration	Import of a JSON export or of a Postman collection
Monitoring	Call Logs	Call history, filtered on the last 7 days by default

2 Installation

2.1 Prerequisites

The Python libraries `requests` and `jsonschema` must be available in the Odoo environment. They are listed in the module's `requirements.txt`:

```
pip install -r iro_api_connector/requirements.txt
```

2.2 Install the module

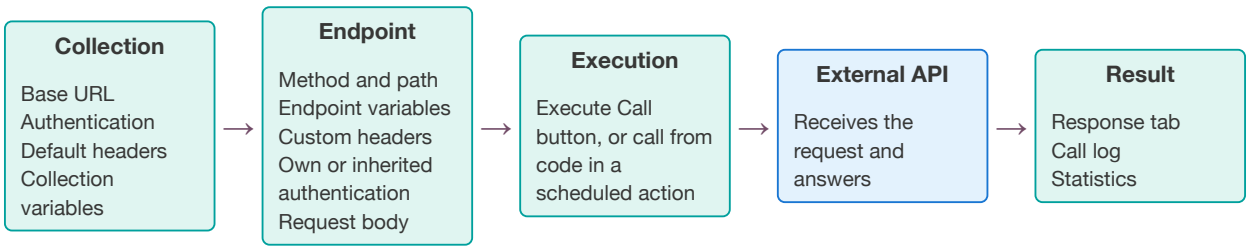
1. Copy the `iro_api_connector` folder into a directory of the addons path (`addons_path`).
2. Enable developer mode, then **Apps > Update Apps List**.
3. Search for **API Connector** and click **Activate**.

2.3 What the installation adds

- The **API Connector** application and its menus.
- Sample collections and endpoints calling public APIs (JSONPlaceholder, httpbin.org, api-adresse.data.gouv.fr, Rick and Morty GraphQL). They serve as templates and can be archived.
- A daily scheduled action, **API Connector: Cleanup Old Logs**, which deletes log entries older than 30 days.

3 Getting Started: a First Call

The diagram below sums up what a call is made of.



The steps below create a collection pointing at httpbin.org, a public service that echoes back what it receives: the response shows exactly the headers and body sent by Odoo.

1. **API Connector > Configuration > Collections > New**. Name: `Partner API (demo)`, Base URL: `https://httpbin.org`.
2. **Authentication** tab: **Bearer Token**, token `demo-token-2026`.
3. **Headers and Variables** tab: default headers `{"Accept": "application/json"}`, collection variables `{"api_version": "v2"}`. Save.
4. **Add Endpoint** button. Name: `Get contact`, method `GET`, path `/anything/{api_version}/contacts/{contact_id}`, endpoint variables `{"contact_id": "42"}`. The full URL shown under the name becomes `https://httpbin.org/anything/v2/contacts/42`. Save.
5. Click **Execute Call**, then open the **Response** tab.

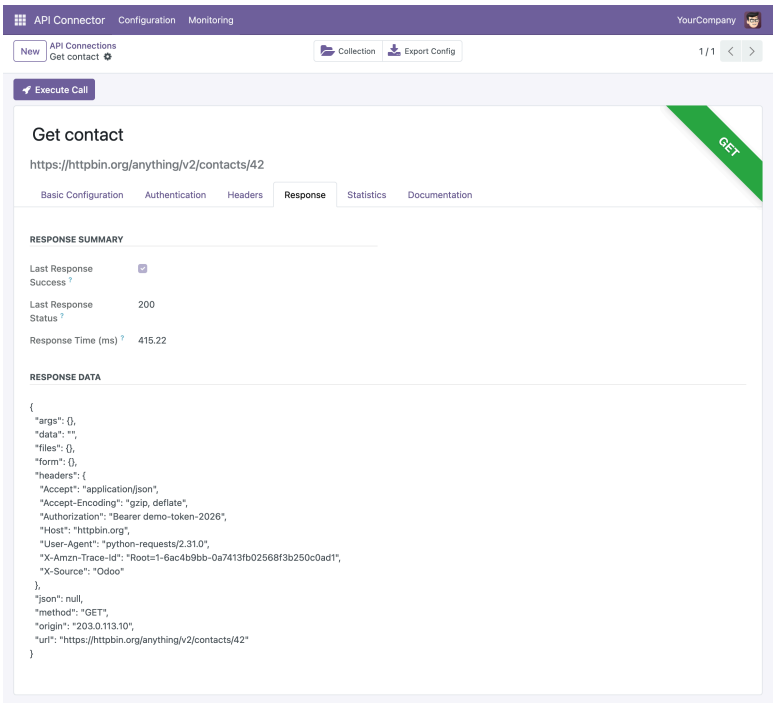


Figure 1: httpbin.org response: the `Authorization` header comes from the collection, the URL received both variables.

The call also appears in **Monitoring > Call Logs**, and the counters of the **Statistics** tab went up by one.

4 Collections

A collection groups the endpoints of one API. Its settings are the defaults of all its endpoints.

4.1 Collection Settings

Field	Purpose
Collection Name	Required.
Description	Free text under the name.
Base URL	Common beginning of the addresses, for example `https://api.example.com`. Used by the endpoints that tick **Use Collection URL**.

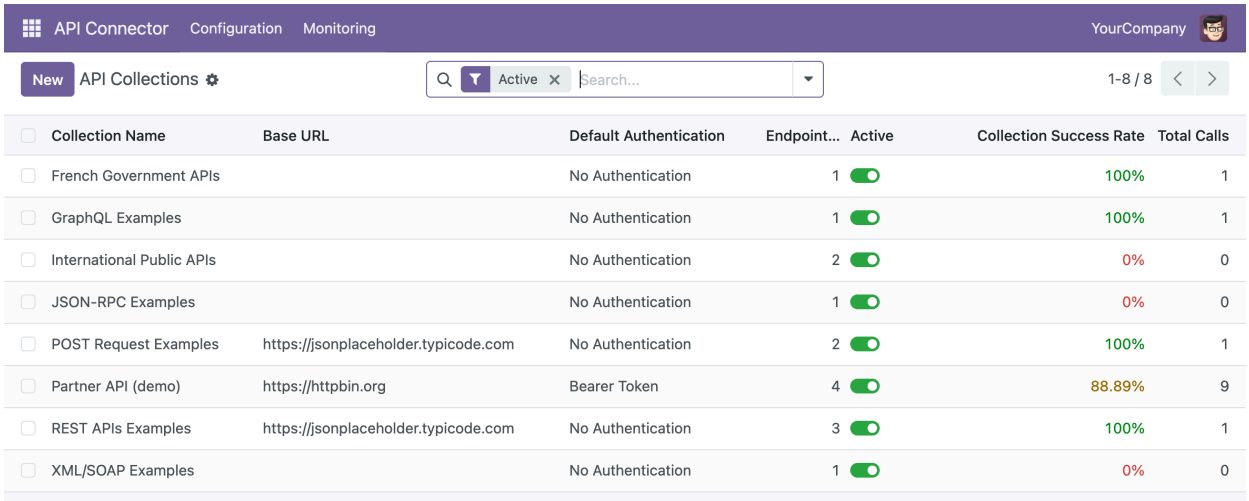


Figure 2: Collection list: base URL, default authentication, number of endpoints, success rate and total calls.

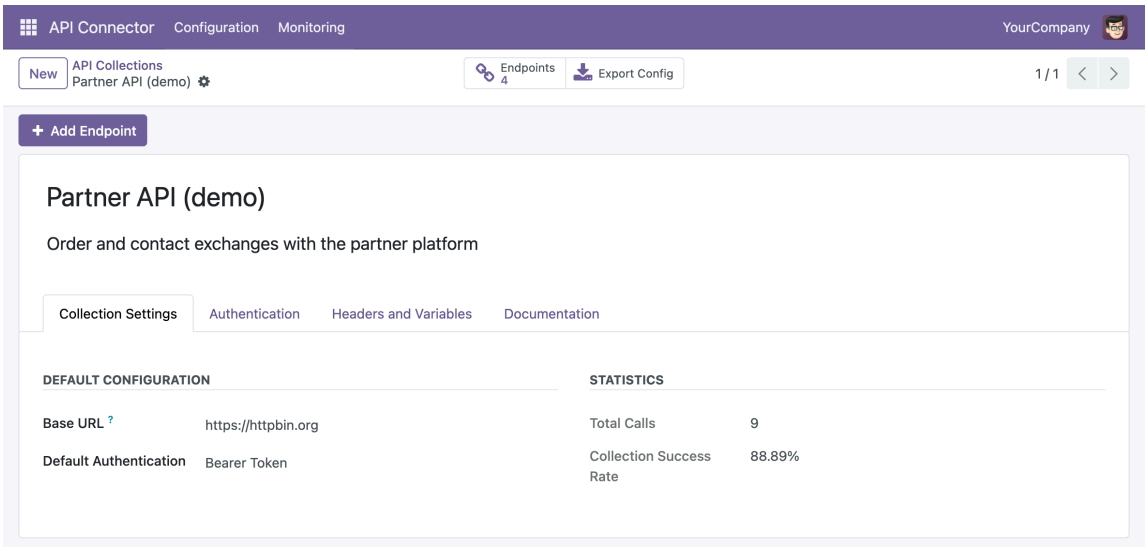


Figure 3: Collection form, Collection Settings tab.

Default Authentication	Used by the endpoints whose authentication is **Inherit from Collection**. Details in the Authentication chapter.
Total Calls, success rate	Sum of the counters of the collection's endpoints.

Form buttons:

- **Add Endpoint** opens an endpoint prefilled with the base URL, authentication and default headers of the collection.
- **Endpoints** shows the list of the collection's endpoints.
- **Export Config** downloads the collection and its endpoints as JSON (see the Export and Import chapter).

Archiving a collection archives all its endpoints. Unarchiving it reactivates them.

4.2 Headers and Variables

- **Default Headers**: JSON object sent with every call of the collection's endpoints.
- **Collection Variables**: JSON object whose values replace the `{name}` placeholders in the endpoints' paths.

Both fields must hold valid JSON: saving is refused otherwise, and the variables must be an object (`{...}`), not a list.

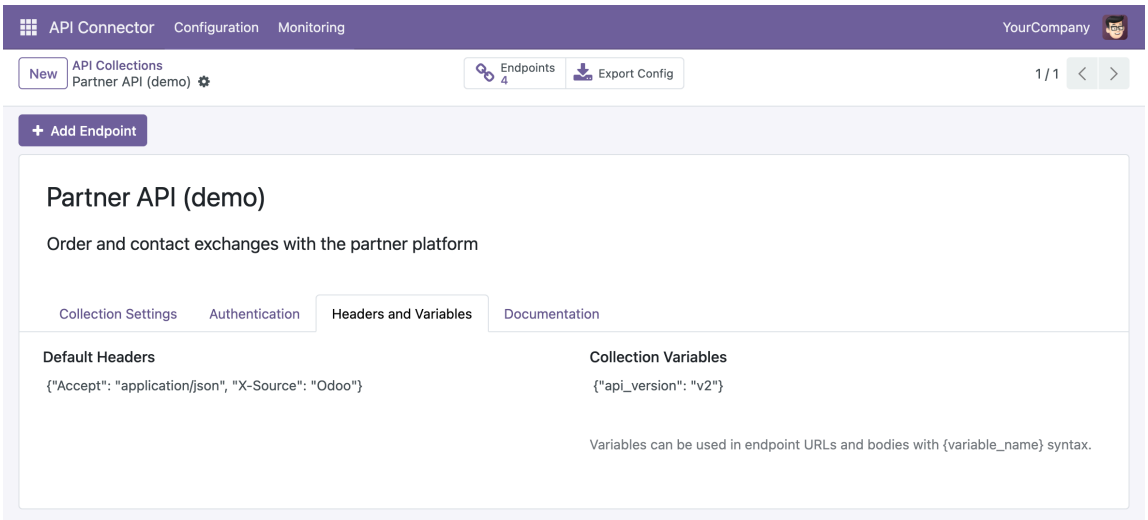


Figure 4: Headers and Variables tab: two JSON objects.

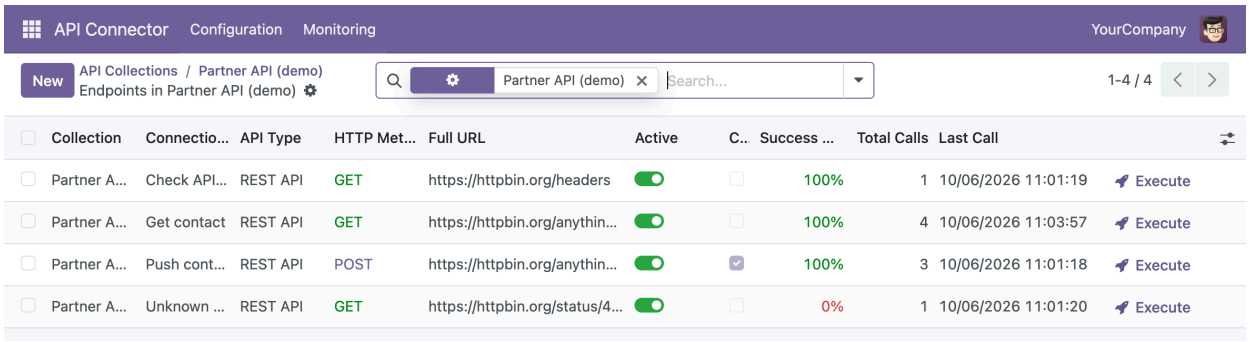


Figure 5: Endpoints of a collection. The C. (critical) column flags the ones protected against deletion.

5 Endpoints

An endpoint describes a call. The list shows them all, with the method in colour, the success rate and an **Execute** button on each line.

5.1 Basic Configuration

Field	Purpose
Collection	Parent collection, optional.
API Type	REST, JSON-RPC, GraphQL or SOAP/XML. For JSON-RPC, GraphQL and SOAP/XML the method switches to POST and can no longer be changed. JSON-RPC also sets the body to JSON.
HTTP Method	GET, POST, PUT, PATCH or DELETE, editable in REST.
Use Collection URL	Shown once a collection is set. Ticked, the base URL comes from the collection. Unticked, the endpoint has its own base URL.
Base URL	Must start with `http://` or `https://`.
Endpoint Path	Appended to the base URL. May contain `{name}` placeholders.

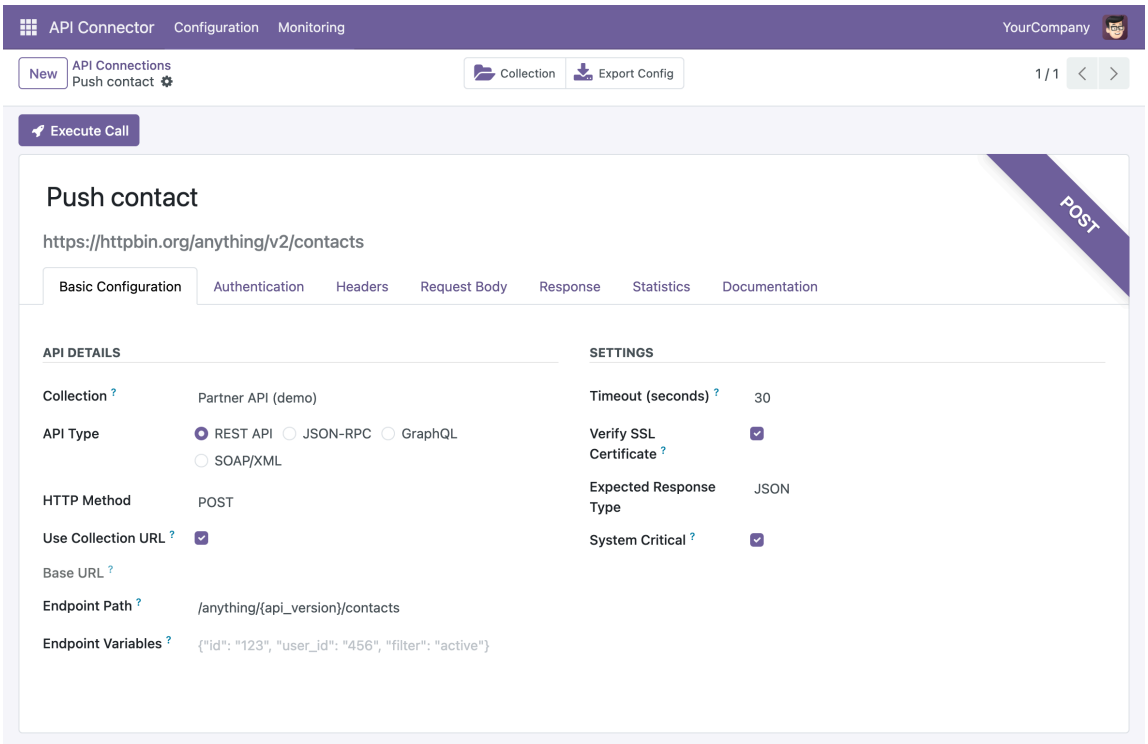


Figure 6: Basic Configuration tab. The ribbon shows the method, the full URL is computed under the name.

Endpoint Variables	JSON object. Adds to the collection variables and wins over them on a shared name.
Timeout (seconds)	From 1 to 300, 30 by default.
Verify SSL Certificate	Ticked by default.
Expected Response Type	JSON, XML, plain text or binary. Sets how the response is read (see Response).
System Critical	Prevents deletion of the endpoint. It can still be disabled with the Active field.

The **Full URL** is the base URL followed by the path, once collection and endpoint variables are replaced. It is recomputed on every change and it is the address called.

5.2 Headers

The **Headers** tab takes a JSON object of custom headers. When sending, Odoo builds the headers in this order, each step winning over the previous one:

1. default headers of the collection;
2. custom headers of the endpoint;
3. authentication header (`Authorization` or the API key header);
4. `Content-Type` derived from the body type, only when not already given: `application/json`, `application/xml`, `application/x-www-form-urlencoded` or `application/octet-stream`.

5.3 Request Body

The tab is hidden for a REST call in GET or DELETE.

Body type	Sent
No Body	No body.
JSON	The text entered, checked as valid JSON at call time.
Form Data (urlencoded)	The key/value pairs of the **Form Data** field, entered as a JSON object.

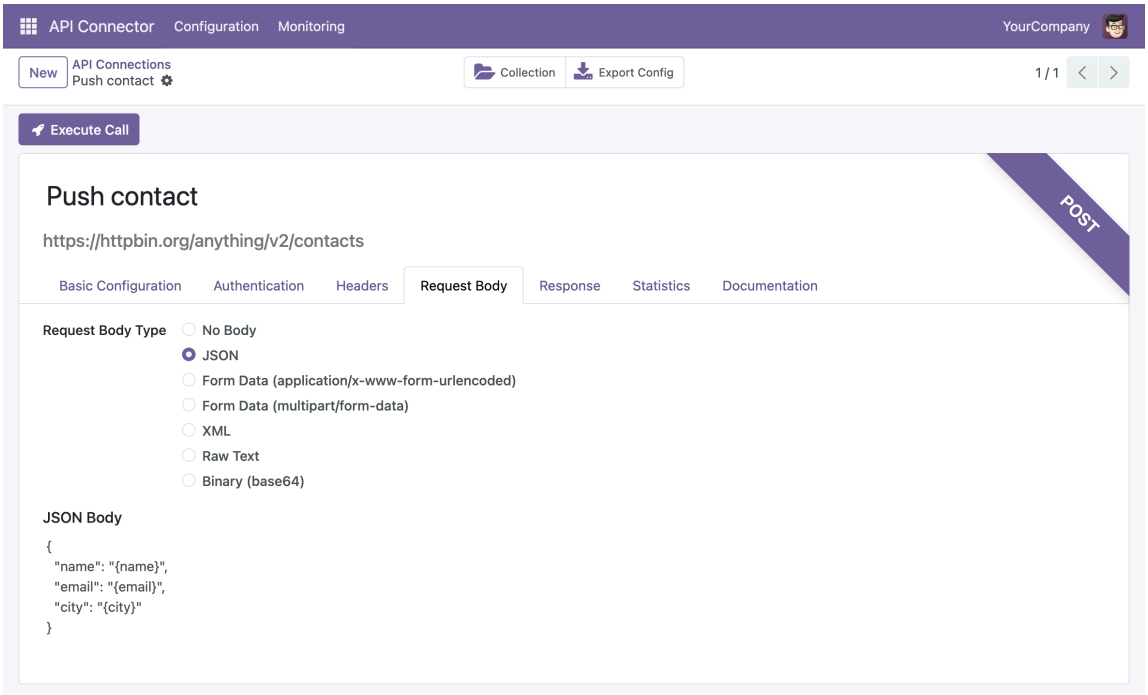


Figure 7: JSON body whose placeholders are filled at execution time.

Form Data (multipart)	The key/value pairs of the **Form Data** field, encoded as `x-www-form-urlencoded` like the previous type.
XML, Raw Text	The text entered, as is.
Binary (base64)	The base64 content entered, decoded before sending.

The `{name}` placeholders of the body and of the form data are replaced by the data passed at execution time (see the Variables chapter).

5.4 Response and Statistics

The **Response** tab shows the result of the last call: success, HTTP status, response time in milliseconds and data received. A JSON response is shown indented. With the **Binary** expected response type, the data is kept as base64.

A call succeeds when the HTTP status is below 400. A 4xx or 5xx status, a timeout or a connection error count as a failure. On a connection error or a timeout no data is received: the error message is read in the log, **Error Details** tab.

The **Statistics** tab gives the total calls, the successful calls, the success rate and the date of the last call.

6 Authentication

6.1 Inherit or define

An endpoint uses **Inherit from Collection** by default: it then uses its collection's authentication. It can also define its own, which takes precedence.

6.2 Methods

Method	Fields	Sent

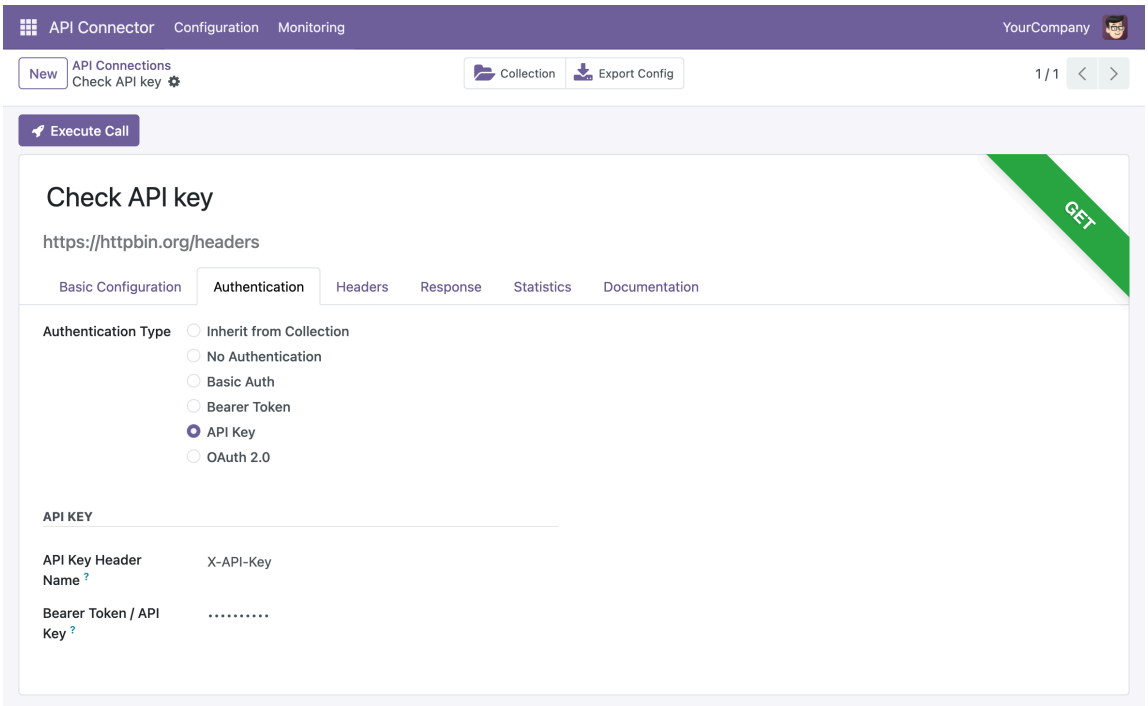


Figure 8: An endpoint replacing its collection's Bearer token with an API key.

No Authentication		Nothing.
Basic Auth	Username, password	HTTP Basic authentication.
Bearer Token	Bearer Token / API Key	`Authorization: Bearer <token>`
API Key	API Key Header Name (`X-API-Key` by default), Bearer Token / API Key	`<header name>: <key>`
OAuth 2.0	Client ID, client secret, token URL, scope	`Authorization: Bearer <access token>`

Password, token and secret fields are masked on screen.

6.3 OAuth 2.0

The **Get New Access Token** button requests a token from the token URL with the **client credentials** flow (`grant_type=client_credentials`, with the client ID, secret and scope). The access token, refresh token and expiry date are stored, and a notification shows the result.

- On a collection, the token obtained is also used by the endpoints that inherit the authentication.
- The module does not renew the token when it expires: click the button again, or paste a token into the field provided under the tokens.

7 Variables

A placeholder is written `{name}`. Variables come from three sources, which do not apply in the same places:

Source	Where it applies	Priority
Collection variables	URL path	Lowest
Endpoint variables	URL path	Wins over the collection
Data passed to `execute_call`	Request body and form data	Only source for the body

Headers do not receive variables. A placeholder without a value stays as is in the request.

Example: with the collection variable `{"api_version": "v2"}`, the path `/anything/{api_version}/contacts` and the body `{"name": "{name}"}`, the call

```
connector.execute_call({'name': 'Addison Olson'})
```

sends `POST https://httpbin.org/anything/v2/contacts` with the body `{"name": "Addison Olson"}`.

8 Automatic Execution

8.1 From a scheduled action

The code of a scheduled action or a server action calls `execute_call` on the endpoint, passing the body values.

```
connector = env['api.connector'].browse(31)
partners = env['res.partner'].search([('email', '!=', False)], limit=3)
```

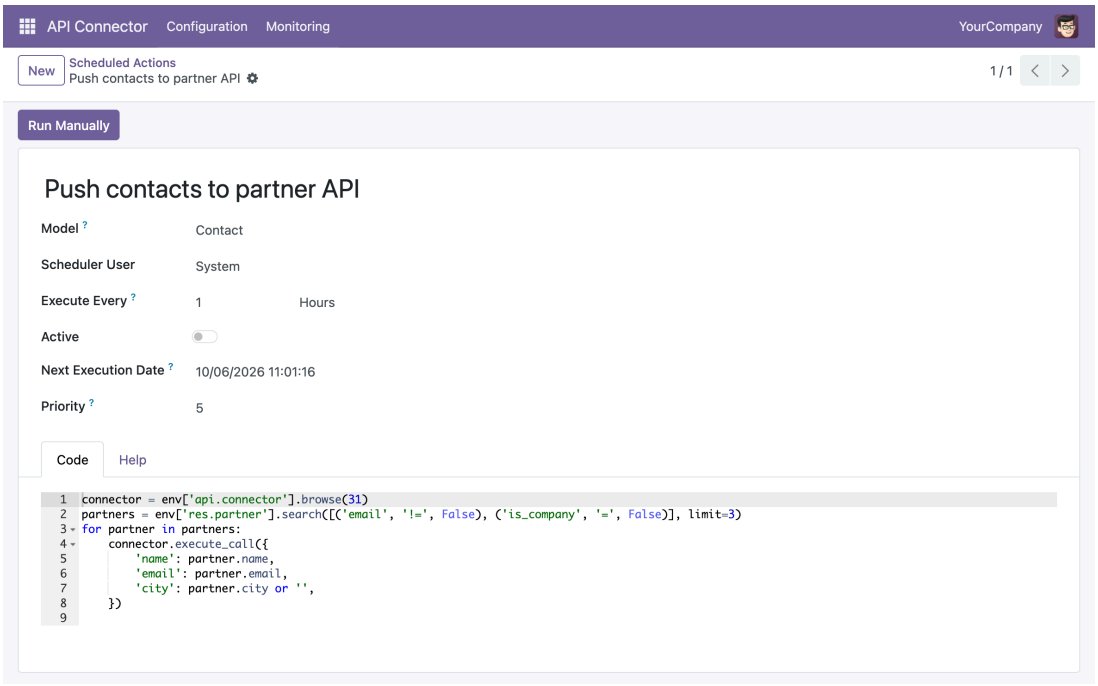


Figure 9: Scheduled action sending three contacts: one call per contact, each with its own values.

```
for partner in partners:
    connector.execute_call({
        'name': partner.name,
        'email': partner.email,
        'city': partner.city or '',
    })
```

The endpoint ID is shown in its **Documentation** tab.

8.2 Reading the result

`execute_call` returns `True` and does not raise when the API answers with a failure: the result is then read on the endpoint.

Field	Content
`last_response_success`	True if the HTTP status is below 400
`last_response_status`	HTTP status, 0 without a response
`last_response_data`	Data received, as text (indented JSON for a JSON response)
`last_response_error`	Error message, empty on success
`last_response_time`	Duration in milliseconds

```
connector.execute_call({'name': partner.name})
if connector.last_response_success:
    data = json.loads(connector.last_response_data)
```

`execute_call` raises an error if the endpoint is archived, or if the JSON or base64 body is not valid once the variables are replaced.

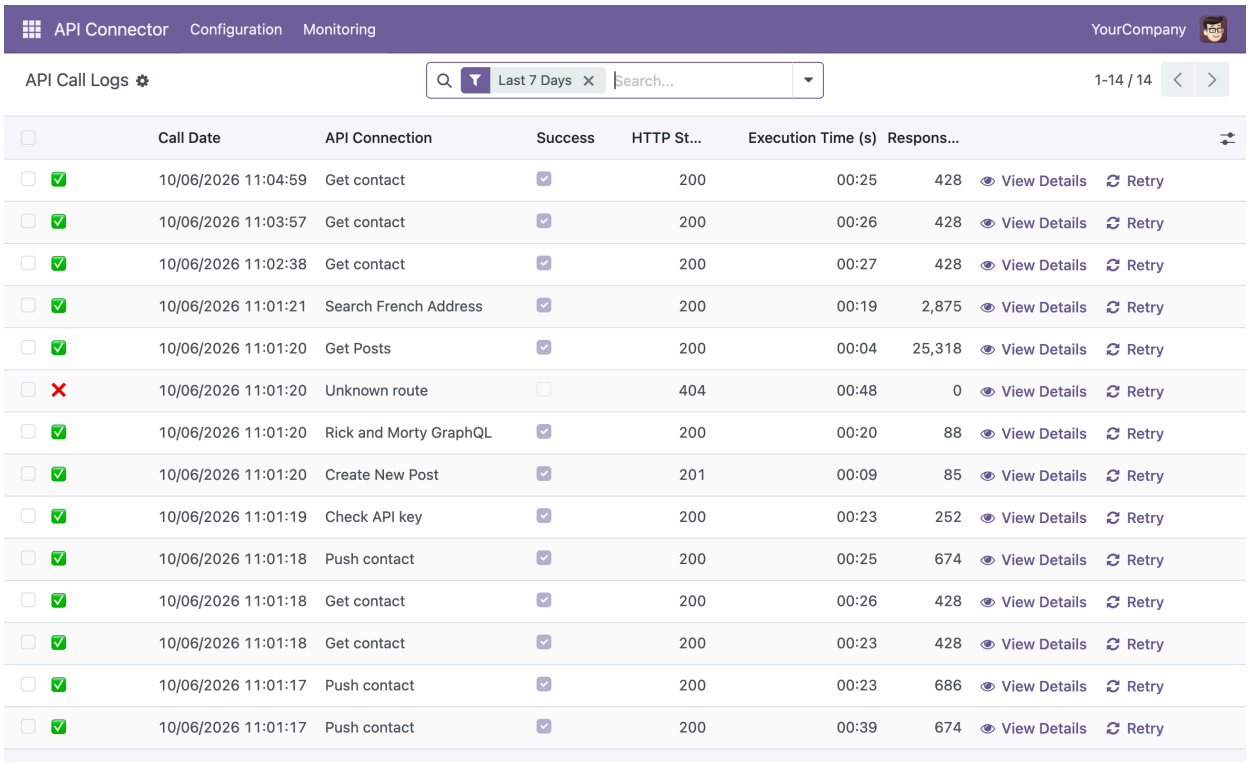


Figure 10: Call log: the unknown route returns a 404 and counts as a failure.

9 Monitoring

9.1 Call log

Monitoring > Call Logs lists every call: status, date, endpoint, success, HTTP code, duration and response size. Filters keep only successes or failures, and group by endpoint, status, code or day.

The details of a call show the data passed at execution time, the data received and, for a failure, the error message.

The **Retry** button runs the call again with the same execution data. The new call is added to the log.

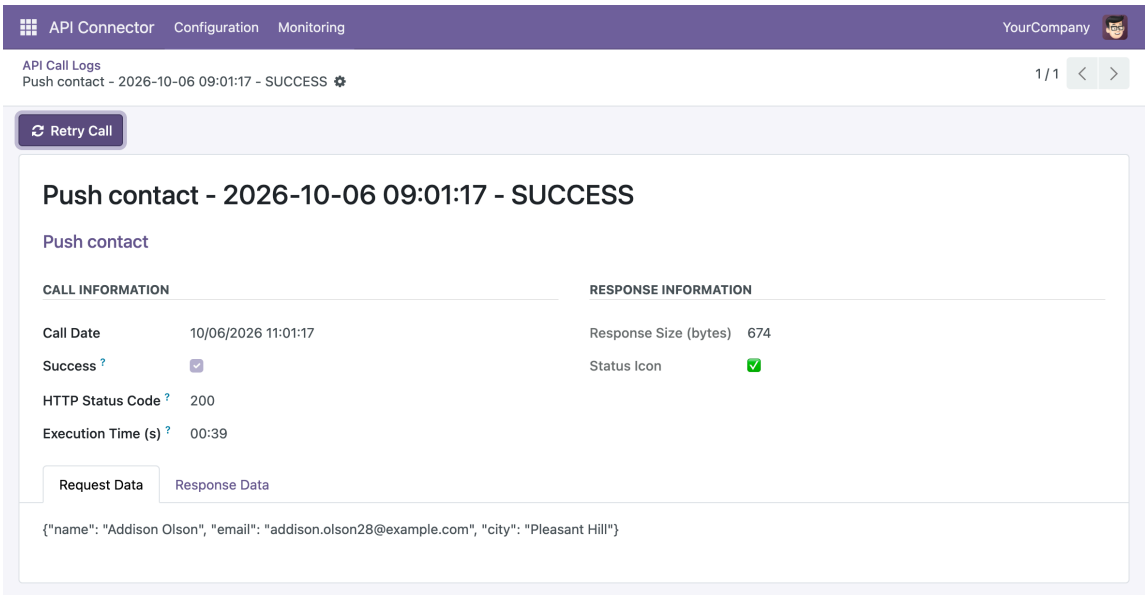


Figure 11: Details of a call made by the scheduled action: the values sent for this contact.

9.2 Retention

The scheduled action **API Connector: Cleanup Old Logs** runs every day and deletes entries older than 30 days. The duration is set in the code of the related server action (`model.cleanup_old_logs(days=30)`).

10 Export and Import

10.1 Export

The **Export Config** button of an endpoint or a collection downloads a JSON file. A collection export contains its endpoints.

> **Warning:** the exported file contains the credentials entered: passwords, tokens, API keys and OAuth secrets. Store and share it like a password.

10.2 Import

Configuration > Import Configuration opens the import wizard.

Configuration Type	Effect
Auto-detect	Recognises the type from the file content.
API Connector	Creates an endpoint, in the chosen **Target Collection** or, failing that, in the collection named as in the export if it exists.
API Collection	Creates the collection and its endpoints.
Postman Collection	Converts a Postman collection.

Import always creates new records: it does not update an existing configuration.

10.3 Postman collections

Accepted versions: v2.0, v2.1 and v2.2 (Postman **Collection v2.1** export).

Postman element	Result in Odoo

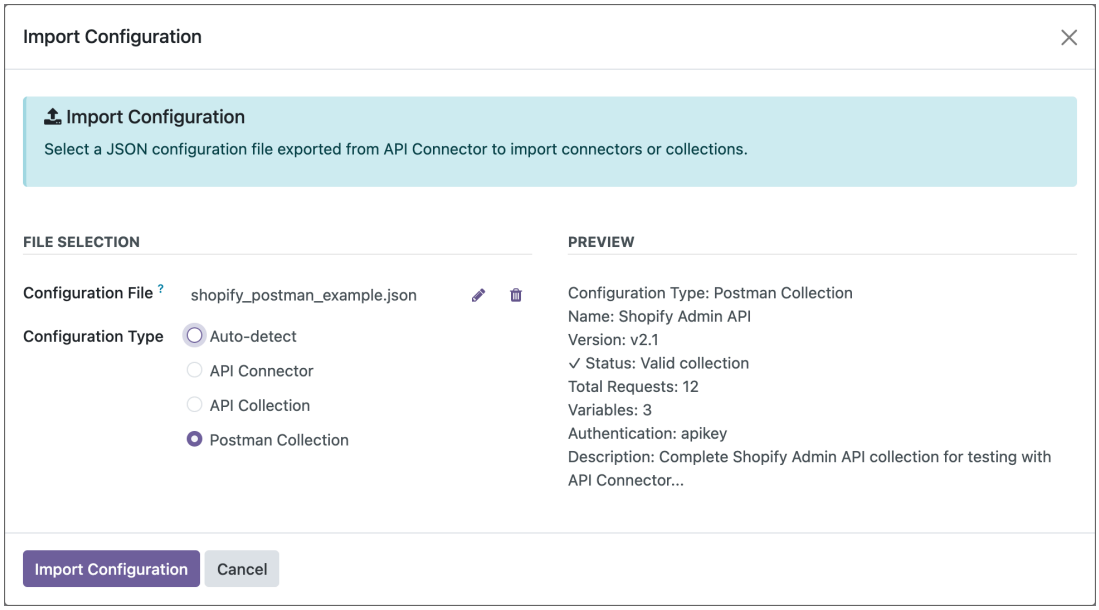


Figure 12: Import wizard: the Postman collection is recognised and summed up before import.

Collection	A collection, with its variables and authentication.
Folders	Flattened: the folder name prefixes the request name (`Products - Get All Products`).
Request	A REST endpoint, with its method, path and headers.
`{{variable}}`	Converted to `{variable}` in the path and the body.
Query parameters and path variables	Endpoint variables. A path variable gets the temporary value `{var_name}`.
Raw body	JSON when valid, raw text otherwise.
GraphQL body	JSON body `{"query": ..., "variables": ...}`.
Basic, bearer, apikey authentication	Imported with its values.

The wizard refuses a collection containing scripts (pre-request or test), NTLM, Hawk, AWS Signature or EdgeGrid authentication, a file body, certificates or proxy settings. A request whose body is `form-data` or `x-www-form-urlencoded` makes the import fail: remove it from the collection before exporting, then create it in Odoo.

After the import:

1. fill in the collection's base URL, which the import leaves empty;
2. replace the temporary `{var_name}` values of the endpoint variables, which win over the collection variables of the same name;
3. enter the real credentials: a token written `{{variable}}` in Postman is imported as is.

11 Access Rights

Group	Rights
Internal user	Read, create, edit and delete collections and endpoints. Read the log.
Administration (Settings)	All rights, the log included.

The module also creates the **API Connector User** and **API Connector Manager** groups. The rights above do not depend on these groups.

Credentials (passwords, tokens, keys, secrets) are stored in the database and masked on screen. Any internal user who opens an endpoint can execute it.

12 Technical Reference

12.1 Models

Model	Purpose
`api.collection`	Collection and shared settings
`api.connector`	Endpoint, execution and last response
`api.call.log`	One line per call
`import.config.wizard`	Import wizard
`postman.parser`	Postman collection conversion (abstract model)

12.2 Useful methods

Model	Method	Effect
`api.connector`	`execute_call(dynamic_data)`	Runs the call with the optional `dynamic_data`, logs it, updates the statistics and the last response. Returns `True`.
`api.connector`	`get_oauth_token()`	Requests an OAuth 2.0 token (client credentials).
`api.collection`	`get_oauth_token()`	Same at collection level.
`api.connector`, `api.collection`	`export_config()`	Creates the JSON attachment of the export.
`api.call.log`	`cleanup_old_logs(days=30)`	Deletes entries older than `days` days.
`api.call.log`	`get_statistics(connector_id, days)`	For one endpoint (or all) over `days` days (7 by default), returns a dictionary: total, successes, failures, success rate, average duration.

12.3 Deleting a critical endpoint

Deleting an endpoint flagged **System Critical** is refused. From code, it goes through with the `force_delete_critical` context: `connector.with_context(force_delete_critical=True).unlink()`.